
BRAWL STARS

Patrones de Diseño de Software

Aplicados al Proyecto Brawl Stars

Alumno

Angel Jesús Moreno Corona

Materia: Diseño de Software

Semestre: 2025-2026

Fecha: Mayo 2026

Patrones: Singleton • Factory Method • Abstract Factory • Builder • Observer • Strategy •

Decorator • Facade

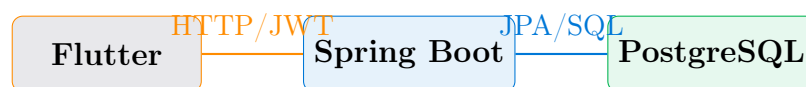
Índice

1. Introducción	2
2. Patrón Singleton (Creacional)	3
2.1. Problema que resuelve	3
2.2. Implementación en el proyecto	3
2.3. Diagrama	4
2.4. Ventajas en el proyecto	4
3. Patrón Factory Method (Creacional)	5
3.1. Problema que resuelve	5
3.2. Implementación en el proyecto	5
3.3. Tipos de productos en el proyecto	6
4. Patrón Observer (Comportamiento)	7
4.1. Problema que resuelve	7
4.2. Implementación en el proyecto	7
4.3. Flujo del Observer en la app	8
5. Patrón Strategy (Comportamiento)	9
5.1. Problema que resuelve	9
5.2. Implementación en el proyecto	9
5.3. Estrategias de autenticación en el proyecto	10
6. Patrón Decorator (Estructural)	11
6.1. Problema que resuelve	11
6.2. Implementación en el proyecto	11
7. Patrón Facade (Estructural)	13
7.1. Problema que resuelve	13
7.2. Implementación en el proyecto	13
7.3. Subsistemas ocultos por la Facade	14
8. Patrón Abstract Factory (Creacional)	15
8.1. Problema que resuelve	15
8.2. Implementación propuesta	15
8.3. Diagrama de la Abstract Factory	18
8.4. Ventajas de implementar este patrón	18
9. Patrón Builder (Creacional)	19
9.1. Problema que resuelve	19
9.2. Implementación propuesta	19
9.3. Diagrama del patrón Builder	22
9.4. Ventajas de implementar este patrón	22
10. Resumen y Conclusiones	23
10.1. Tabla comparativa de patrones	23
10.2. Conclusiones	24

1 Introducción

Mi documento presenta el análisis y aplicación de **ocho patrones de diseño** de software sobre el proyecto **Brawl Stars**, una aplicación desarrollada con *Spring Boot* (backend), *Flutter* (frontend móvil/web) y una base de datos *PostgreSQL* alojada en Aiven.

El proyecto consiste en una red social temática donde los jugadores pueden publicar, reaccionar y comentar sobre **brawlers** (personajes del juego Brawl Stars de Supercell). El sistema implementa autenticación JWT, roles de usuario y operaciones CRUD completas.



Los patrones analizados pertenecen a las tres categorías clásicas del libro *Gang of Four*:

bsDark!15 Categoría	Patrones Aplicados
Creacionales	Singleton, Factory Method, Abstract Factory, Builder
Estructurales	Decorator, Facade
De Comportamiento	Observer, Strategy

2 Patrón Singleton (Creacional)

Definición

El patrón **Singleton** garantiza que una clase tenga *una sola instancia* y proporciona un punto de acceso global a ella. Se utiliza para recursos que deben compartirse en toda la aplicación.

2.1 Problema que resuelve

En el proyecto Brawl Stars, la conexión a la base de datos es un recurso costoso. Si cada operación creara una nueva conexión, se agotarían los recursos del servidor rápidamente.

2.2 Implementación en el proyecto

El archivo `DatabaseConnection.java` encontrado en la carpeta `creational/singleton/` implementa este patrón directamente:

```
1 public class DatabaseConnection {
2
3     // Instancia nica del Singleton
4     private static DatabaseConnection instance;
5
6     // Objeto conexi n
7     private Connection connection;
8
9     // Datos de conexi n (credenciales reales del proyecto)
10    private static final String URL = "jdbc:mysql://localhost:3306/
11    mi_base";
12    private static final String USER = "angelCor";
13    private static final String PASSWORD = "angelMor";
14
15    // Constructor privado: impide instanciaci n externa
16    private DatabaseConnection() {
17        try {
18            Class.forName("com.mysql.cj.jdbc.Driver");
19            connection = DriverManager.getConnection(URL, USER,
20            PASSWORD);
21            System.out.println("Conexi n establecida");
22        } catch (Exception e) {
23            throw new RuntimeException("Error al conectar a la base de
24            datos", e);
25        }
26    }
27
28    // M todo de acceso global -- solo crea la instancia UNA vez
29    public static DatabaseConnection getInstance() {
30        if (instance == null) {
31            instance = new DatabaseConnection();
32        }
33        return instance;
34    }
35
36    // Obtiene la conexi n activa
```

```

34     public Connection getConnection() {
35         return connection;
36     }
37 }

```

Listing 1: DatabaseConnection.java – Singleton de conexión a base de datos

En el backend Spring Boot, este patrón se extiende a nivel de framework: los **Beans de Spring** (servicios, repositorios, controladores) son singletons por defecto:

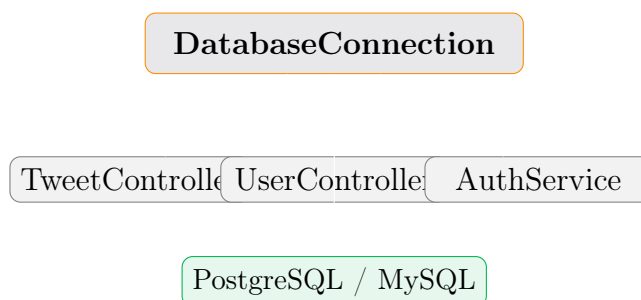
```

1  @Component // Spring lo crea UNA sola vez y lo reutiliza
2  public class JwtUtils {
3      @Value("${bezkoder.app.jwtSecret}")
4      private String jwtSecret;
5
6      @Value("${bezkoder.app.jwtExpirationMs}")
7      private int jwtExpirationMs;
8
9      public String generateJwtToken(Authentication authentication) {
10         UserDetailsImpl userPrincipal =
11             (UserDetailsImpl) authentication.getPrincipal();
12
13         return Jwts.builder()
14             .setSubject((userPrincipal.getUsername()))
15             .setIssuedAt(new Date())
16             .setExpiration(new Date((new Date()).getTime() +
17                 jwtExpirationMs))
18             .signWith(key(), SignatureAlgorithm.HS256)
19             .compact();
20     }
21 }

```

Listing 2: JwtUtils.java – Bean Singleton gestionado por Spring

2.3 Diagrama



2.4 Ventajas en el proyecto

- Evita múltiples conexiones innecesarias a la base de datos.
- El JwtUtils se crea una sola vez: todos los controladores comparten la misma instancia del validador de tokens.
- Reduce el consumo de memoria en el servidor Render.

3 Patrón Factory Method (Creacional)

Definición

El **Factory Method** define una interfaz para crear objetos, pero deja que las subclases decidan qué clase instanciar. Permite *delegar la creación* de objetos a métodos especializados.

3.1 Problema que resuelve

El proyecto maneja diferentes tipos de respuestas HTTP según la operación (login exitoso, error de autenticación, datos de brawler). En lugar de construir estas respuestas directamente en los controladores, el Factory Method centraliza su creación.

3.2 Implementación en el proyecto

Los **DTOs de respuesta** actúan como productos del factory. Spring utiliza internamente este patrón para los *ResponseEntity*:

```
1 public class TweetResponseDTO {
2     private Long id;
3     private String nombre;           // Nombre del Brawler
4     private String ataquePrincipal;
5     private String rareza;
6     private String urlImagen;
7     private int cantidadLikes;
8     private boolean likedByCurrentUser;
9
10    // Constructor de la factory
11    public TweetResponseDTO(Long id, String nombre, String
12        ataquePrincipal,
13        String rareza, String urlImagen,
14        int cantidadLikes, boolean
15        likedByCurrentUser) {
16        this.id = id;
17        this.nombre = nombre;
18        this.ataquePrincipal = ataquePrincipal;
19        this.rareza = rareza;
20        this.urlImagen = urlImagen;
21        this.cantidadLikes = cantidadLikes;
22        this.likedByCurrentUser = likedByCurrentUser;
23    }
24 }
```

Listing 3: TweetResponseDTO.java – Objeto producto de la factory

```
1 @PostMapping("/signin")
2 public ResponseEntity<?> authenticateUser(
3     @RequestBody LoginRequest loginRequest) {
4
5     Authentication authentication = authManager.authenticate(
6         new UsernamePasswordAuthenticationToken(
7             loginRequest.getUsername(),
8             loginRequest.getPassword())
9     );
```

```

10
11 String jwt = jwtUtils.generateJwtToken(authentication);
12 UserDetailsImpl userDetails =
13     (UserDetailsImpl) authentication.getPrincipal();
14
15 // Factory: crea el objeto de respuesta seg n el tipo de usuario
16 return ResponseEntity.ok(new JwtResponse(jwt,
17     userDetails.getId(),
18     userDetails.getUsername(),
19     userDetails.getEmail(),
20     roles)); // <-- JwtResponse es el "producto" de la factory
21 }

```

Listing 4: AuthController.java – Uso del patrón Factory para respuestas

3.3 Tipos de productos en el proyecto

bsDark!15Factory	Producto	Uso
ResponseEntity	JwtResponse	Login exitoso
ResponseEntity	MessageResponse	Mensajes de error/éxito
ResponseEntity	TweetResponseDTO	Datos de brawlers
ResponseEntity	UserResponseDTO	Datos del usuario

4 Patrón Observer (Comportamiento)

Definición

El **Observer** define una dependencia uno-a-muchos entre objetos: cuando un objeto cambia de estado, todos sus dependientes son notificados automáticamente.

4.1 Problema que resuelve

En Brawl Stars (la app), cuando un usuario da **like** a un brawler o agrega un **comentario**, múltiples partes del sistema deben reaccionar: la base de datos se actualiza, la UI muestra el cambio, y eventualmente podría notificarse a otros usuarios.

4.2 Implementación en el proyecto

El sistema de reacciones (TweetReaction) sigue el patrón Observer. La entidad Tweet (brawler) es el **sujeto observado**:

```
1 @Entity
2 @Table(name = "tweets")
3 public class Tweet {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @NotBlank
10    private String nombre;           // Nombre del brawler
11
12    @NotBlank
13    private String ataquePrincipal;
14
15    @NotBlank
16    private String rareza;
17
18    @NotBlank
19    private String urlImagen;
20
21    // Lista de observadores (usuarios que dieron like)
22    @ManyToMany(fetch = FetchType.LAZY)
23    @JoinTable(
24        name = "tweet_likes",
25        joinColumns = @JoinColumn(name = "tweet_id"),
26        inverseJoinColumns = @JoinColumn(name = "user_id")
27    )
28    private Set<User> likes = new HashSet<>();
29 }
```

Listing 5: Tweet.java – El sujeto observado con su lista de likes

```
1 @RestController
2 @RequestMapping("/api/reactions")
3 public class TweetReactionController {
4
5     @Autowired
```



```

6     private TweetRepository tweetRepo;
7
8     @PostMapping("/{tweetId}/like")
9     public ResponseEntity<?> toggleLike(
10         @PathVariable Long tweetId,
11         Authentication auth) {
12
13         Tweet tweet = tweetRepo.findById(tweetId)
14             .orElseThrow(() -> new RuntimeException("Brawler no
15             encontrado"));
16
17         UserDetailsImpl userDetails =
18             (UserDetailsImpl) auth.getPrincipal();
19
20         // "Notificar" el cambio: agregar/quitar el like
21         if (tweet.getLikes().contains(userDetails.getId())) {
22             tweet.getLikes().remove(userDetails.getId()); // quitar
23             like
24         } else {
25             tweet.getLikes().add(userDetails.getId()); // agregar
26             like
27         }
28
29         tweetRepo.save(tweet); // Persistir el nuevo estado
30
31         return ResponseEntity.ok(new MessageResponse(
32             "Reacci'on actualizada correctamente"));
33     }
34 }

```

Listing 6: TweetReactionController.java – El controlador notifica cambios

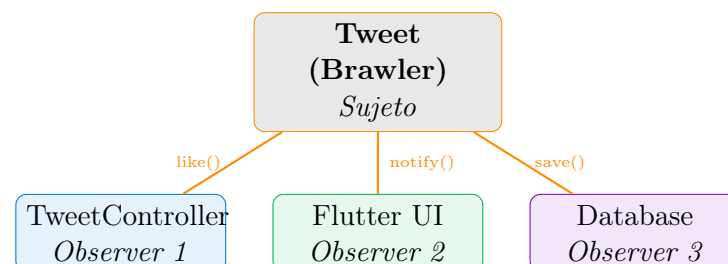
```

1 public enum EReaction {
2     LIKE,
3     LOVE,
4     HAHA,
5     WOW,
6     SAD,
7     ANGRY
8 }

```

Listing 7: EReaction.java – Tipos de reacciones disponibles

4.3 Flujo del Observer en la app



5 Patrón Strategy (Comportamiento)

Definición

El **Strategy** define una familia de algoritmos, los encapsula y los hace intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo usan.

5.1 Problema que resuelve

El sistema de autenticación necesita diferentes **estrategias de seguridad**: validar por JWT, validar por rol, o permitir acceso público. En lugar de poner toda la lógica en un solo lugar, cada estrategia se encapsula por separado.

5.2 Implementación en el proyecto

Spring Security implementa el patrón Strategy nativamente. `WebSecurityConfig.java` define qué estrategia de autenticación usar para cada ruta:

```
1 @Configuration
2 @EnableMethodSecurity
3 public class WebSecurityConfig {
4
5     @Autowired
6     UserDetailsServiceImpl userDetailsService; // Estrategia de carga
7         de usuario
8
9     @Autowired
10    private AuthEntryPointJwt unauthorizedHandler; // Estrategia de
11        error 401
12
13    // Estrategia de filtrado JWT
14    @Bean
15    public AuthTokenFilter authenticationJwtTokenFilter() {
16        return new AuthTokenFilter();
17    }
18
19    @Bean
20    public SecurityFilterChain filterChain(HttpSecurity http) throws
21        Exception {
22        http
23            .exceptionHandling(
24                e -> e.authenticationEntryPoint(unauthorizedHandler)
25            )
26            .sessionManagement(
27                s -> s.sessionCreationPolicy(SessionCreationPolicy.
28                    STATELESS)
29            )
30            .authorizeHttpRequests(auth -> auth
31                // Estrategia 1: Acceso p blico (sin autenticaci n)
32                .requestMatchers("/api/auth/**").permitAll()
33                .requestMatchers("/api/test/**").permitAll()
34                // Estrategia 2: Requiere JWT v lido
35                .anyRequest().authenticated()
36            );
37    }
```

```

33
34      // Aplica la estrategia JWT antes de cualquier verificaci n
35      http.addFilterBefore(authenticationJwtTokenFilter(),
36                          UsernamePasswordAuthenticationFilter.class);
37
38      return http.build();
39  }
40 }

```

Listing 8: WebSecurityConfig.java – Definición de estrategias de seguridad

```

1  @Service
2  public class UserDetailsServiceImpl implements UserDetailsService {
3
4      @Autowired
5      UserRepository userRepository;
6
7      // Estrategia: buscar usuario en base de datos por username
8      @Override
9      @Transactional
10     public UserDetails loadUserByUsername(String username)
11         throws UsernameNotFoundException {
12
13         User user = userRepository.findByUsername(username)
14             .orElseThrow(() -> new UsernameNotFoundException(
15                 "Usuario no encontrado: " + username));
16
17         return UserDetailsImpl.build(user);
18     }
19 }

```

Listing 9: UserDetailsServiceImpl.java – Estrategia de carga de usuarios

5.3 Estrategias de autenticación en el proyecto

bsDark!15Estrategia	Clase	Aplicación
JWT Token	AuthTokenFilter	Rutas protegidas
Acceso público	permitAll()	/api/auth/**
Carga de usuario	UserDetailsServiceImpl	Login
Manejo de error	AuthEntryPointJwt	Error 401

6 Patrón Decorator (Estructural)

Definición

El **Decorator** adjunta responsabilidades adicionales a un objeto de forma dinámica. Ofrece una alternativa flexible a la herencia para extender funcionalidad.

6.1 Problema que resuelve

En el proyecto, los datos de un **Tweet** (brawler) necesitan enriquecerse antes de enviarse al cliente Flutter: se añade el conteo de likes, si el usuario actual ya dio like, y los comentarios. El Decorator agrega estos datos sin modificar la entidad original.

6.2 Implementación en el proyecto

El **TweetResponseDTO** actúa como un decorator del **Tweet** original, añadiendo información adicional:

```
1 @Entity
2 @Table(name = "tweets")
3 public class Tweet {
4     private Long id;
5     private String nombre;
6     private String ataquePrincipal;
7     private String rareza;
8     private String urlImagen;
9     private Set<User> likes = new HashSet<>();
10    // Solo datos crudos de la base de datos
11 }
```

Listing 10: Tweet.java – Entidad base (sin decorar)

```
1 public class TweetResponseDTO {
2     // Datos originales del Tweet
3     private Long id;
4     private String nombre;
5     private String ataquePrincipal;
6     private String rareza;
7     private String urlImagen;
8
9     // Datos DECORADOS (agregados dinámicamente)
10    private int cantidadLikes;           // Calculado en tiempo real
11    private boolean likedByCurrentUser;  // Personalizado por usuario
12    private List<String> comentarios;    // Enriquecido con
13                                         comentarios
14
15    // Constructor que "decora" el tweet original
16    public static TweetResponseDTO fromTweet(
17        Tweet tweet, Long currentUserId) {
18
19        TweetResponseDTO dto = new TweetResponseDTO();
20        dto.id = tweet.getId();
21        dto.nombre = tweet.getNombre();
22        dto.ataquePrincipal = tweet.getAtaquePrincipal();
23        dto.rareza = tweet.getRareza();
```

```
23     dto.urlImagen = tweet.getUrlImagen();
24
25     // Decoración: agregar conteo y estado de like
26     dto.cantidadLikes = tweet.getLikes().size();
27     dto.likedByCurrentUser = tweet.getLikes().stream()
28         .anyMatch(u -> u.getId().equals(currentUserId));
29
30     return dto;
31 }
32 }
```

Listing 11: TweetResponseDTO.java – Decorator: Tweet enriquecido

```
1 @GetMapping
2 public ResponseEntity<List<TweetResponseDTO>> getAllTweets(
3     Authentication auth) {
4
5     UserDetailsImpl userDetails =
6         (UserDetailsImpl) auth.getPrincipal();
7
8     List<Tweet> tweets = tweetRepo.findAll();
9
10    // Decorar cada tweet con información adicional
11    List<TweetResponseDTO> response = tweets.stream()
12        .map(t -> TweetResponseDTO.fromTweet(t, userDetails.getId()))
13        .collect(Collectors.toList());
14
15    return ResponseEntity.ok(response);
16 }
```

Listing 12: TweetController.java – Aplica la decoración al recuperar brawlers

7 Patrón Facade (Estructural)

Definición

El **Facade** proporciona una interfaz simplificada a un subsistema complejo. Oculta la complejidad interna y expone solo lo necesario al cliente.

7.1 Problema que resuelve

El proceso de **registro e inicio de sesión** involucra múltiples subsistemas: validación de datos, verificación de usuario existente, cifrado de contraseña, asignación de rol, generación de JWT y respuesta HTTP. El *AuthService* hace de Facade, simplificando todo esto en una sola llamada.

7.2 Implementación en el proyecto

```
1 @Service
2 public class AuthService {
3
4     @Autowired AuthenticationManager authManager;
5     @Autowired UserRepository userRepo;
6     @Autowired RoleRepository roleRepo;
7     @Autowired PasswordEncoder encoder;
8     @Autowired JwtUtils jwtUtils;
9
10    // Facade: oculta toda la complejidad del registro
11    public MessageResponse registerUser(SignupRequest request) {
12
13        // 1. Validar si username ya existe
14        if (userRepo.existsByUsername(request.getUsername())) {
15            return new MessageResponse("Error: El usuario ya existe");
16        }
17
18        // 2. Validar si email ya existe
19        if (userRepo.existsByEmail(request.getEmail())) {
20            return new MessageResponse("Error: El email ya est en
21            uso");
22        }
23
24        // 3. Crear el usuario con contrase a encriptada
25        User user = new User(
26            request.getUsername(),
27            request.getEmail(),
28            encoder.encode(request.getPassword()) // BCrypt
29        );
30
31        // 4. Asignar rol por defecto
32        Role userRole = roleRepo.findByName(ERole.ROLE_USER)
33            .orElseThrow(() -> new RuntimeException("Rol no encontrado
34            "));
35        user.setRoles(Set.of(userRole));
36
37        // 5. Guardar en base de datos
38        userRepo.save(user);
39    }
40 }
```

```
37     return new MessageResponse("Usuario registrado exitosamente");
38 }
39
40 // Facade: oculta toda la complejidad del login
41 public JwtResponse loginUser(LoginRequest request) {
42     Authentication auth = authManager.authenticate(
43         new UsernamePasswordAuthenticationToken(
44             request.getUsername(), request.getPassword())
45     );
46
47     SecurityContextHolder.getContext().setAuthentication(auth);
48     String jwt = jwtUtils.generateJwtToken(auth);
49
50     UserDetailsImpl user = (UserDetailsImpl) auth.getPrincipal();
51
52     return new JwtResponse(jwt, user.getId(),
53         user.getUsername(), user.getEmail(), user.getRoles());
54 }
55 }
56 }
```

Listing 13: AuthService.java – Facade del sistema de autenticación

```
1 @RestController
2 @RequestMapping("/api/auth")
3 public class AuthController {
4
5     @Autowired
6     AuthService authService; // Solo conoce la Facade
7
8     @PostMapping("/signup")
9     public ResponseEntity<?> registerUser(
10         @RequestBody SignupRequest req) {
11         // Una sola llamada oculta toda la complejidad
12         return ResponseEntity.ok(authService.registerUser(req));
13     }
14
15     @PostMapping("/signin")
16     public ResponseEntity<?> authenticateUser(
17         @RequestBody LoginRequest req) {
18         // Una sola llamada oculta: validaci n, JWT, roles...
19         return ResponseEntity.ok(authService.loginUser(req));
20     }
21 }
```

Listing 14: AuthController.java – El cliente solo ve la Facade simple

7.3 Subsistemas ocultos por la Facade

8 Patrón Abstract Factory (Creacional)

Definición

El **Abstract Factory** proporciona una interfaz para crear *familias de objetos relacionados* sin especificar sus clases concretas. Es una fábrica de fábricas.

Nota de implementación

Este patrón fue **propuesto como mejora** al proyecto Brawl Stars. Aunque el código actual no lo implementa explícitamente, se ideó cómo podría integrarse en el sistema de notificaciones y creación de perfiles de brawler para distintas plataformas (móvil y web).

8.1 Problema que resuelve

El proyecto necesita crear **familias de objetos de respuesta** que varían según la plataforma del cliente (Flutter móvil vs. Flutter web). Un brawler mostrado en móvil tiene una resolución de imagen diferente, campos resumidos y paginación distinta respecto a la vista web. El Abstract Factory centraliza esta lógica.

8.2 Implementación propuesta

La idea es tener una **fábrica abstracta** `BrawlerResponseFactory` con dos implementaciones concretas: una para móvil y otra para web.

```

1 // Interfaz de la Abstract Factory
2 public interface BrawlerResponseFactory {
3
4     // Crea la respuesta de un brawler individual
5     BrawlerDTO crearBrawlerDTO(Tweet tweet, Long userId);
6
7     // Crea la respuesta de una lista de brawlers (paginada)
8     BrawlerListDTO crearListaDTO(List<Tweet> tweets,
9                                   Long userId, int pagina);
10
11     // Crea la respuesta de comentarios asociados
12     ComentarioDTO crearComentarioDTO(TweetComment comentario);
13 }

```

Listing 15: `BrawlerResponseFactory.java` – Interfaz de la Abstract Factory

```

1 // Producto concreto: versión móvil (imágenes comprimidas, datos
  resumidos)
2 @Component("mobileFactory")
3 public class MobileBrawlerFactory implements BrawlerResponseFactory {
4
5     @Override
6     public BrawlerDTO crearBrawlerDTO(Tweet tweet, Long userId) {
7         BrawlerDTO dto = new BrawlerDTO();
8         dto.setId(tweet.getId());
9         dto.setNombre(tweet.getNombre());
10        dto.setAtaque(tweet.getAtaquePrincipal());
11        dto.setRareza(tweet.getRareza());

```



```

12      // Mvil: URL de imagen en baja resoluci n (thumbnail)
13      dto.setUrlImagen(tweet.getUrlImagen() + "?size=150x150");
14      dto.setLikes(tweet.getLikes().size());
15      dto.setLikedByUser(tweet.getLikes().stream()
16          .anyMatch(u -> u.getId().equals(userId)));
17
18      return dto;
19  }
20
21  @Override
22  public BrawlerListDTO crearListaDTO(List<Tweet> tweets,
23      Long userId, int pagina) {
24      // Mvil: paginaci n de 10 elementos
25      int inicio = pagina * 10;
26      List<BrawlerDTO> items = tweets.subList(
27          Math.min(inicio, tweets.size()),
28          Math.min(inicio + 10, tweets.size())
29      ).stream()
30      .map(t -> crearBrawlerDTO(t, userId))
31      .collect(Collectors.toList());
32
33      return new BrawlerListDTO(items, tweets.size(), pagina, 10);
34  }
35
36  @Override
37  public ComentarioDTO crearComentarioDTO(TweetComment c) {
38      // Mvil: comentario resumido (m x 100 chars)
39      String texto = c.getContenido();
40      if (texto.length() > 100)
41          texto = texto.substring(0, 97) + "...";
42      return new ComentarioDTO(c.getId(), c.getAutor(), texto);
43  }
44  }
45  }

```

Listing 16: MobileBrawlerFactory.java – Factory concreta para móvil

```

1  // Producto concreto: versi n web (im genes HD, datos completos)
2  @Component("webFactory")
3  public class WebBrawlerFactory implements BrawlerResponseFactory {
4
5      @Override
6      public BrawlerDTO crearBrawlerDTO(Tweet tweet, Long userId) {
7          BrawlerDTO dto = new BrawlerDTO();
8          dto.setId(tweet.getId());
9          dto.setNombre(tweet.getNombre());
10         dto.setAtaque(tweet.getAtaquePrincipal());
11         dto.setRareza(tweet.getRareza());
12         dto.setBiografia(tweet.getBiografia()); // Campo extra solo
13         en web
14
15         // Web: URL de imagen en alta resoluci n
16         dto.setUrlImagen(tweet.getUrlImagen() + "?size=800x600");
17         dto.setLikes(tweet.getLikes().size());
18         dto.setLikedByUser(tweet.getLikes().stream()
19             .anyMatch(u -> u.getId().equals(userId)));
20
21         return dto;

```

```

21     }
22
23     @Override
24     public BrawlerListDTO crearListaDTO(List<Tweet> tweets,
25                                         Long userId, int pagina) {
26         // Web: paginaci n de 25 elementos
27         int inicio = pagina * 25;
28         List<BrawlerDTO> items = tweets.subList(
29             Math.min(inicio, tweets.size()),
30             Math.min(inicio + 25, tweets.size())
31         ).stream()
32             .map(t -> crearBrawlerDTO(t, userId))
33             .collect(Collectors.toList());
34
35         return new BrawlerListDTO(items, tweets.size(), pagina, 25);
36     }
37
38     @Override
39     public ComentarioDTO crearComentarioDTO(TweetComment c) {
40         // Web: comentario completo con fecha y edici n
41         return new ComentarioDTO(c.getId(), c.getAutor(),
42             c.getContenido(), c.getFecha(), c.isEditado());
43     }
44 }

```

Listing 17: WebBrawlerFactory.java – Factory concreta para web

```

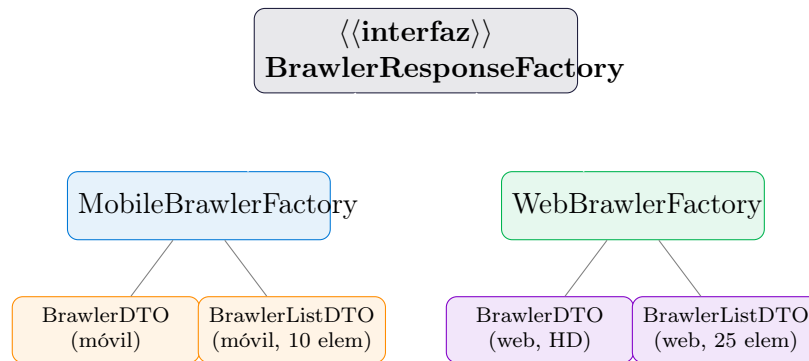
1  @RestController
2  @RequestMapping("/api/tweets")
3  public class TweetController {
4
5      @Autowired
6      @Qualifier("mobileFactory")
7      private BrawlerResponseFactory mobileFactory;
8
9      @Autowired
10     @Qualifier("webFactory")
11     private BrawlerResponseFactory webFactory;
12
13     @GetMapping
14     public ResponseEntity<?> getAllBrawlers(
15         @RequestHeader(value = "X-Platform",
16             defaultValue = "mobile") String platform,
17         @RequestParam(defaultValue = "0") int pagina,
18         Authentication auth) {
19
20         UserDetailsImpl user = (UserDetailsImpl) auth.getPrincipal();
21         List<Tweet> tweets = tweetRepo.findAll();
22
23         // Seleccionar la factory seg n la plataforma del cliente
24         BrawlerResponseFactory factory =
25             platform.equals("web") ? webFactory : mobileFactory;
26
27         // La factory crea la familia de objetos correcta
28         BrawlerListDTO respuesta =
29             factory.crearListaDTO(tweets, user.getId(), pagina);
30
31         return ResponseEntity.ok(respuesta);

```

```
32     }  
33 }
```

Listing 18: TweetController.java – Uso del Abstract Factory según plataforma

8.3 Diagrama de la Abstract Factory



8.4 Ventajas de implementar este patrón

- Permite soportar nuevas plataformas (p.ej., *tablet* o *smart TV*) sin modificar los controladores.
- Garantiza consistencia: todos los objetos de una familia son compatibles entre sí.
- La selección de factory se hace dinámicamente según el header **X-Platform** de la petición HTTP.

9 Patrón Builder (Creacional)

Definición

El **Builder** separa la *construcción* de un objeto complejo de su *representación*, permitiendo que el mismo proceso de construcción pueda crear diferentes representaciones.

Nota de implementación

Este patrón fue **propuesto como mejora** al proyecto Brawl Stars. Se ideó para resolver el problema de crear objetos **Brawler** y **User** con muchos campos opcionales, evitando constructores con decenas de parámetros (el llamado *telescoping constructor antipattern*).

9.1 Problema que resuelve

Al registrar un nuevo **brawler** en la app, el objeto puede tener muchos campos opcionales: biografía, precio en gemas, tipo de recarga, estadísticas de daño, etc. Sin Builder, el constructor se vuelve un caos con múltiples sobrecargas. Con Builder, cada campo se agrega de forma fluida y legible.

9.2 Implementación propuesta

```
1 @Entity
2 @Table(name = "brawlers")
3 public class Brawler {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     // Campos obligatorios
10    private String nombre;
11    private String rareza;
12    private String ataquePrincipal;
13    private String urlImagen;
14
15    // Campos opcionales
16    private String biografia;
17    private Double precioGemas;
18    private String tipoRecarga;
19    private Integer danioMaximo;
20    private Integer vidaMaxima;
21    private String superHabilidad;
22    private String gadget;
23    private String starPower;
24
25    // Constructor privado: solo el Builder puede instanciarlo
26    private Brawler(Builder builder) {
27        this.nombre = builder.nombre;
28        this.rareza = builder.rareza;
29        this.ataquePrincipal = builder.ataquePrincipal;
```

```

30     this.urlImagen      = builder.urlImagen;
31     this.biografia      = builder.biografia;
32     this.precioGemas    = builder.precioGemas;
33     this.tipoRecarga    = builder.tipoRecarga;
34     this.danioMaximo    = builder.danioMaximo;
35     this.vidaMaxima     = builder.vidaMaxima;
36     this.superHabilidad = builder.superHabilidad;
37     this.gadget         = builder.gadget;
38     this.starPower      = builder.starPower;
39 }
40
41 // ===== CLASE BUILDER INTERNA =====
42 public static class Builder {
43
44     // Campos obligatorios
45     private final String nombre;
46     private final String rareza;
47     private final String ataquePrincipal;
48     private final String urlImagen;
49
50     // Campos opcionales con valores por defecto
51     private String biografia = "";
52     private Double precioGemas = 0.0;
53     private String tipoRecarga = "Golpes";
54     private Integer danioMaximo = 0;
55     private Integer vidaMaxima = 0;
56     private String superHabilidad = "";
57     private String gadget = "";
58     private String starPower = "";
59
60     // Constructor del Builder: solo los campos obligatorios
61     public Builder(String nombre, String rareza,
62                   String ataque, String urlImagen) {
63         this.nombre = nombre;
64         this.rareza = rareza;
65         this.ataquePrincipal = ataque;
66         this.urlImagen = urlImagen;
67     }
68
69     // M todos fluidos para campos opcionales
70     public Builder biografia(String bio) {
71         this.biografia = bio; return this;
72     }
73     public Builder precioGemas(Double precio) {
74         this.precioGemas = precio; return this;
75     }
76     public Builder tipoRecarga(String tipo) {
77         this.tipoRecarga = tipo; return this;
78     }
79     public Builder danioMaximo(Integer danio) {
80         this.danioMaximo = danio; return this;
81     }
82     public Builder vidaMaxima(Integer vida) {
83         this.vidaMaxima = vida; return this;
84     }
85     public Builder superHabilidad(String super_) {
86         this.superHabilidad = super_; return this;
87     }

```

```

88     public Builder gadget(String g) {
89         this.gadget = g; return this;
90     }
91     public Builder starPower(String sp) {
92         this.starPower = sp; return this;
93     }
94
95     // M todo terminal: construye el objeto final
96     public Brawler build() {
97         return new Brawler(this);
98     }
99 }
100 }

```

Listing 19: Brawler.java – La entidad con Builder interno

```

1  @Service
2  public class BrawlerService {
3
4      @Autowired
5      private BrawlerRepository brawlerRepo;
6
7      public Brawler crearBrawlerDesdeRequest(BrawlerRequest req) {
8
9          // Uso del Builder: fluido, legible, sin constructores largos
10         Brawler brawler = new Brawler.Builder(
11             req.getNombre(),
12             req.getRareza(),
13             req.getAtaquePrincipal(),
14             req.getUrlImagen()
15         )
16             .biografia(req.getBiografia())
17             .precioGemas(req.getPrecioGemas())
18             .tipoRecarga(req.getTipoRecarga())
19             .danioMaximo(req.getDanioMaximo())
20             .vidaMaxima(req.getVidaMaxima())
21             .superHabilidad(req.getSuperHabilidad())
22             .gadget(req.getGadget())
23             .starPower(req.getStarPower())
24             .build(); // <-- Construye el objeto final
25
26         return brawlerRepo.save(brawler);
27     }
28
29     // Ejemplo: crear un brawler solo con campos obligatorios
30     public Brawler crearBrawlerBasico(String nombre) {
31         Brawler brawler = new Brawler.Builder(
32             nombre, "Comun", "Disparo", "default.png"
33         )
34             .build(); // Los opcionales quedan en sus valores por
35             defecto
36
37         return brawlerRepo.save(brawler);
38     }
39 }

```

Listing 20: BrawlerService.java – Uso del Builder para crear brawlers

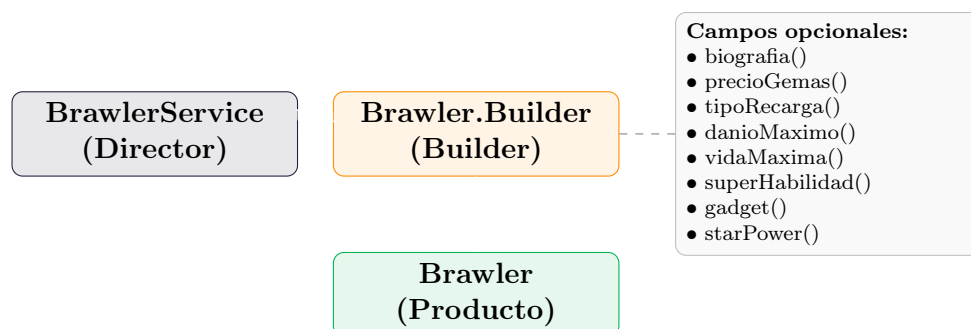
```

1 //      SIN Builder: constructor telesc pico ilegible
2 Brawler shelly = new Brawler("Shelly", "Comun",
3     "Escopeta de balas", "shelly.png",
4     "Shelly es la brawler introductoria del juego...",
5     0.0, "Golpes", 1960, 3360,
6     "Cortina de balas",
7     "Blindaje de arcilla",
8     "Cepo de acero");
9
10 //      CON Builder: fluido, legible y seguro
11 Brawler shelly = new Brawler.Builder(
12     "Shelly", "Comun", "Escopeta de balas", "shelly.png")
13     .biografia("Shelly es la brawler introductoria del juego...")
14     .tipoRecarga("Golpes")
15     .danioMaximo(1960)
16     .vidaMaxima(3360)
17     .superHabilidad("Cortina de balas")
18     .gadget("Blindaje de arcilla")
19     .starPower("Cepo de acero")
20     .build();

```

Listing 21: Ejemplo de creación sin Builder vs. con Builder

9.3 Diagrama del patrón Builder



9.4 Ventajas de implementar este patrón

- Elimina los constructores con múltiples parámetros del mismo tipo (propensos a errores de orden).
- El código que crea brawlers es autoexplicativo y fácil de leer.
- Permite crear brawlers básicos (solo campos obligatorios) o completos (todos los campos) con el mismo Builder.
- Facilita las pruebas unitarias: cada test puede construir exactamente el brawler que necesita.

10 Resumen y Conclusiones

10.1 Tabla comparativa de patrones

bsDark!15 Patrón	Tipo	Clase principal	Beneficio
Singleton	Creacional	DatabaseConnection, JwtUtils	Una sola instancia del recurso
Factory Method	Creacional	TweetResponseDTO, JwtResponse	Creación centralizada de objetos
Abstract Factory	Creacional	BrawlerResponseFactory	Familias de objetos por plataforma
Builder	Creacional	Brawler.Builder	Construcción fluida de objetos complejos
Observer	Comportamiento	Tweet, TweetReactionController	Reacción automática a cambios
Strategy	Comportamiento	WebSecurityConfig, UserDetailsService-Impl	Algoritmos de seguridad intercambiables
Decorator	Estructural	TweetResponseDTO	Enriquecer datos sin modificar la entidad
Facade	Estructural	AuthService	Simplifica subsistemas complejos

10.2 Conclusiones

Conclusiones finales

- Los **patrones de diseño** no son reglas rígidas, sino soluciones probadas que se adaptan al contexto del proyecto.
- En el proyecto Brawl Stars, el uso de Spring Boot facilita la implementación de patrones como **Singleton** y **Strategy** de forma nativa a través del contenedor IoC.
- El patrón **Facade** (AuthService) reduce el acoplamiento entre la capa de presentación y la lógica de negocio.
- El patrón **Observer** permite que el sistema de likes y comentarios sea extensible sin modificar el código existente.
- El **Abstract Factory** propuesto permitiría dar soporte a múltiples plataformas (móvil, web, tablet) sin duplicar la lógica de los controladores.
- El **Builder** propuesto elimina los constructores telescópicos al crear brawlers con muchos campos opcionales, haciendo el código más legible y mantenible.
- Aplicar estos ocho patrones mejora la *mantenibilidad*, *testabilidad* y *escalabilidad* del sistema, factores cruciales cuando el proyecto crece y nuevos brawlers o funcionalidades son añadidos.

Angel Jesús Moreno Corona

Patrones de Diseño de Software – 2026
